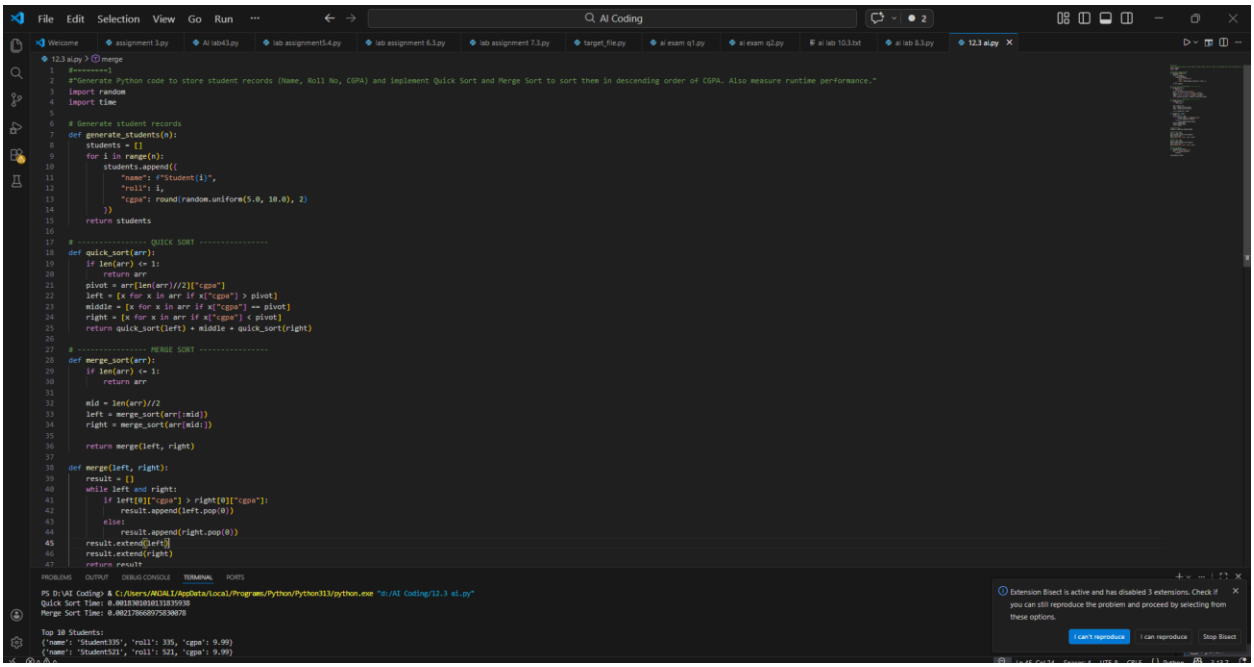
## 2303a51924

## Lab assignment-12.3

**Task 1: Sorting Student Records for Placement Drive**

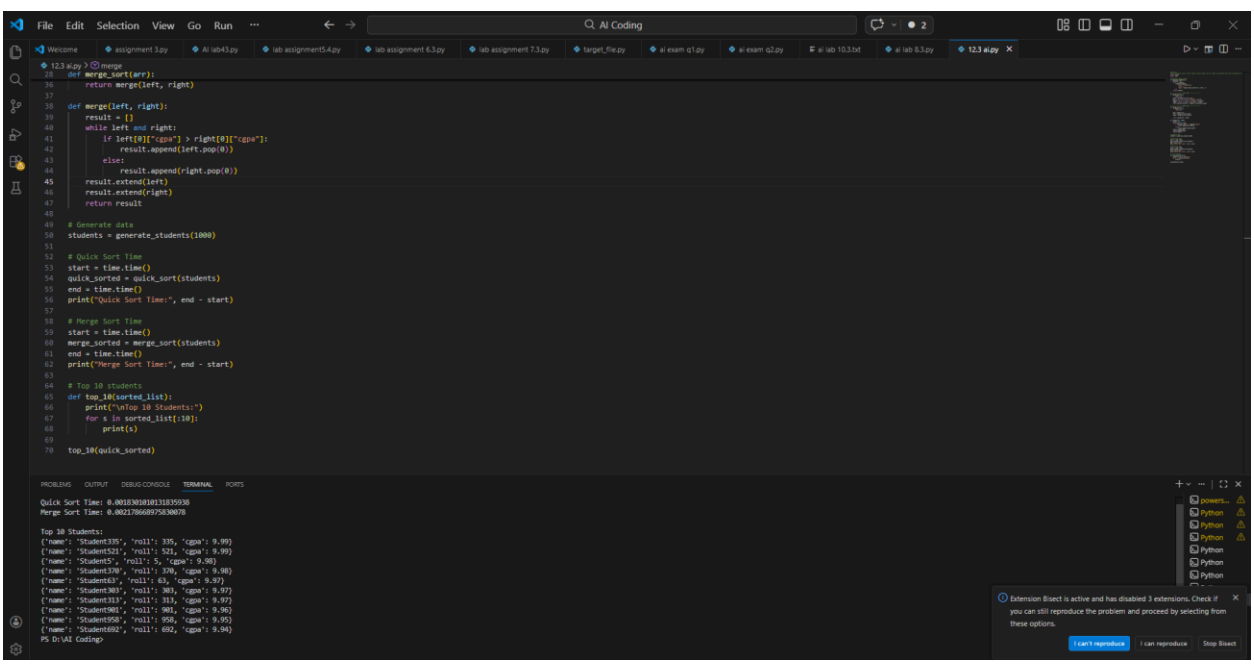
**Scenario**

**SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.**



```python
#=======1
#"Generate Python code to store student records (Name, Roll No, CGPA) and implement Quick Sort and Merge Sort to sort them in descending order of CGPA. Also measure runtime performance."
import random
import time

# Generate student records
def generate_students(n):
    students = []
    for i in range(n):
        students.append({
            "name": f"Student{i}",
            "roll": i,
            "cgpa": round(random.uniform(5.0, 10.0), 2)
        })
    return students

# ---------------- QUICK SORT ----------------
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]["cgpa"]
    left = [x for x in arr if x["cgpa"] > pivot]
    middle = [x for x in arr if x["cgpa"] == pivot]
    right = [x for x in arr if x["cgpa"] < pivot]
    return quick_sort(left) + middle + quick_sort(right)

# ---------------- MERGE SORT ----------------
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    while left and right:
        if left[0]["cgpa"] > right[0]["cgpa"]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))
    result.extend(left)
    result.extend(right)
    return result
```

```
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/12.3 ai.py"
Quick Sort Time: 0.001830101013135938
Merge Sort Time: 0.002178668975830078

Top 10 Students:
{'name': 'Student335', 'roll': 335, 'cgpa': 9.99}
{'name': 'Student521', 'roll': 521, 'cgpa': 9.99}
```

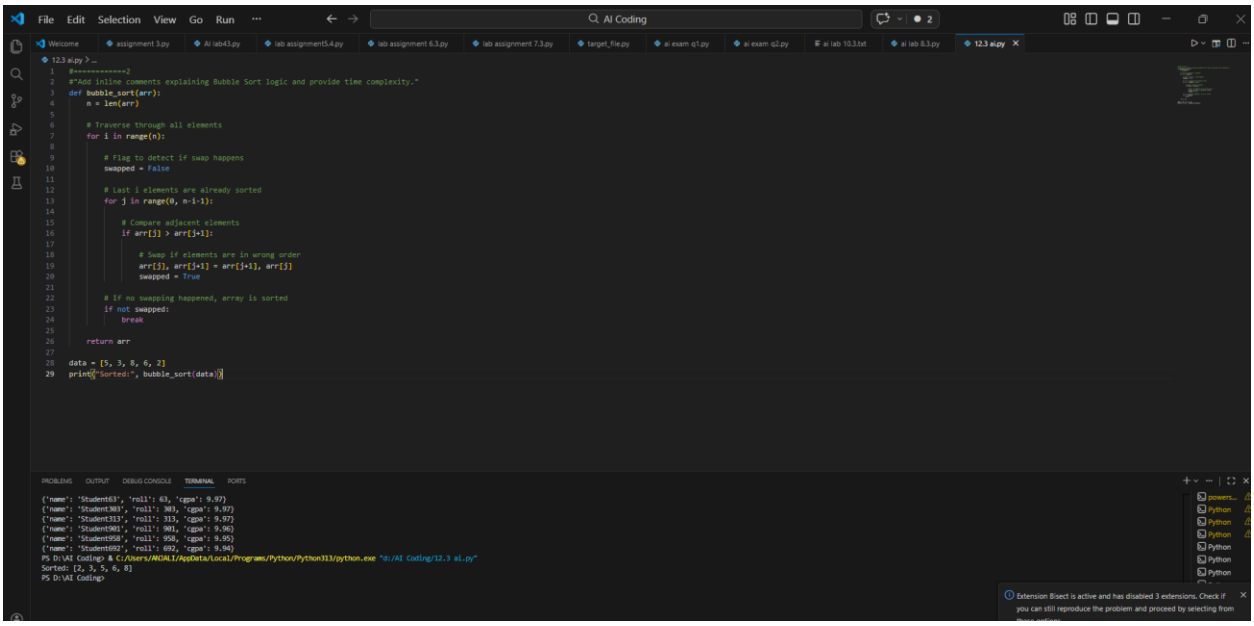


```python
def merge_sort(arr):
    return merge(left, right)

def merge(left, right):
    result = []
    while left and right:
        if left[0]["cgpa"] > right[0]["cgpa"]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))
    result.extend(left)
    result.extend(right)
    return result

# Generate data
students = generate_students(1000)

# Quick Sort Time
start = time.time()
quick_sorted = quick_sort(students)
end = time.time()
print("Quick Sort Time:", end - start)

# Merge Sort Time
start = time.time()
merge_sorted = merge_sort(students)
end = time.time()
print("Merge Sort Time:", end - start)

# Top 10 students
def top_10(sorted_list):
    print("\nTop 10 Students:")
    for s in sorted_list[:10]:
        print(s)

top_10(quick_sorted)
```

```
Quick Sort Time: 0.001830101013135938
Merge Sort Time: 0.002178668975830078

Top 10 Students:
{'name': 'Student335', 'roll': 335, 'cgpa': 9.99}
{'name': 'Student521', 'roll': 521, 'cgpa': 9.99}
{'name': 'Student5', 'roll': 5, 'cgpa': 9.98}
{'name': 'Student370', 'roll': 370, 'cgpa': 9.98}
{'name': 'Student63', 'roll': 63, 'cgpa': 9.97}
{'name': 'Student303', 'roll': 303, 'cgpa': 9.97}
{'name': 'Student313', 'roll': 313, 'cgpa': 9.97}
{'name': 'Student901', 'roll': 901, 'cgpa': 9.96}
{'name': 'Student958', 'roll': 958, 'cgpa': 9.95}
{'name': 'Student692', 'roll': 692, 'cgpa': 9.94}
PS D:\AI Coding>
```

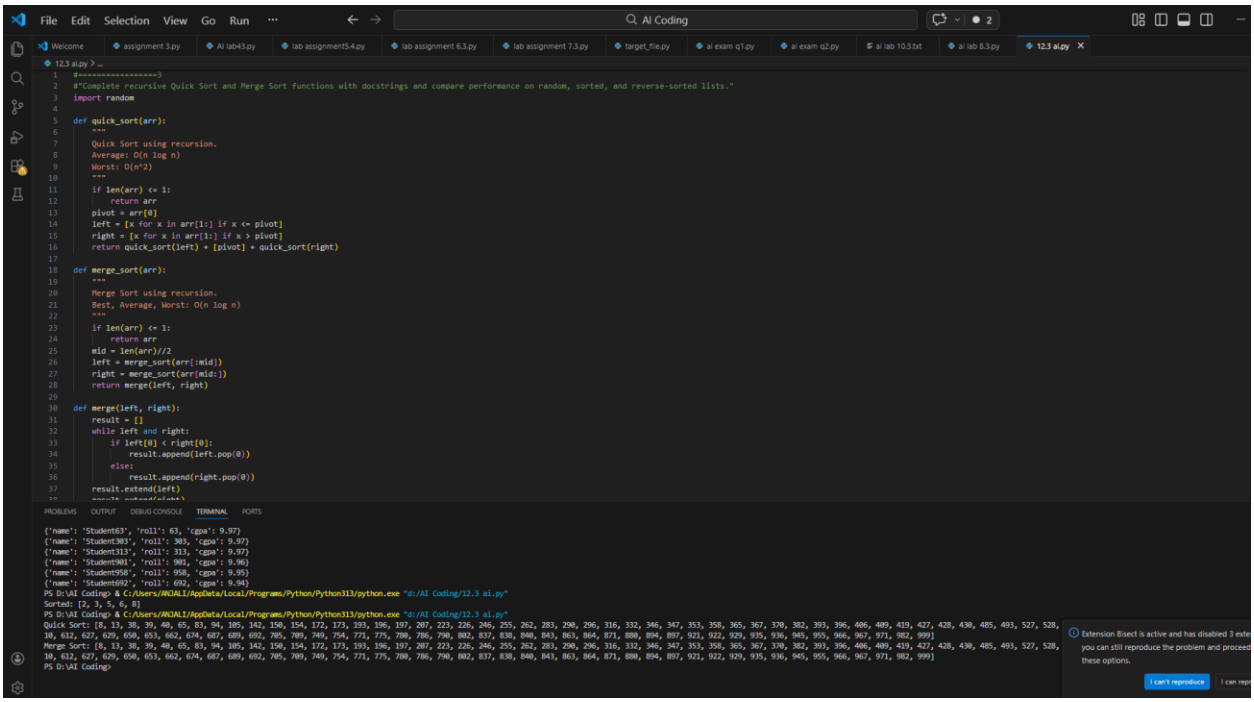
**Task 2: Implementing Bubble Sort with AI Comments**

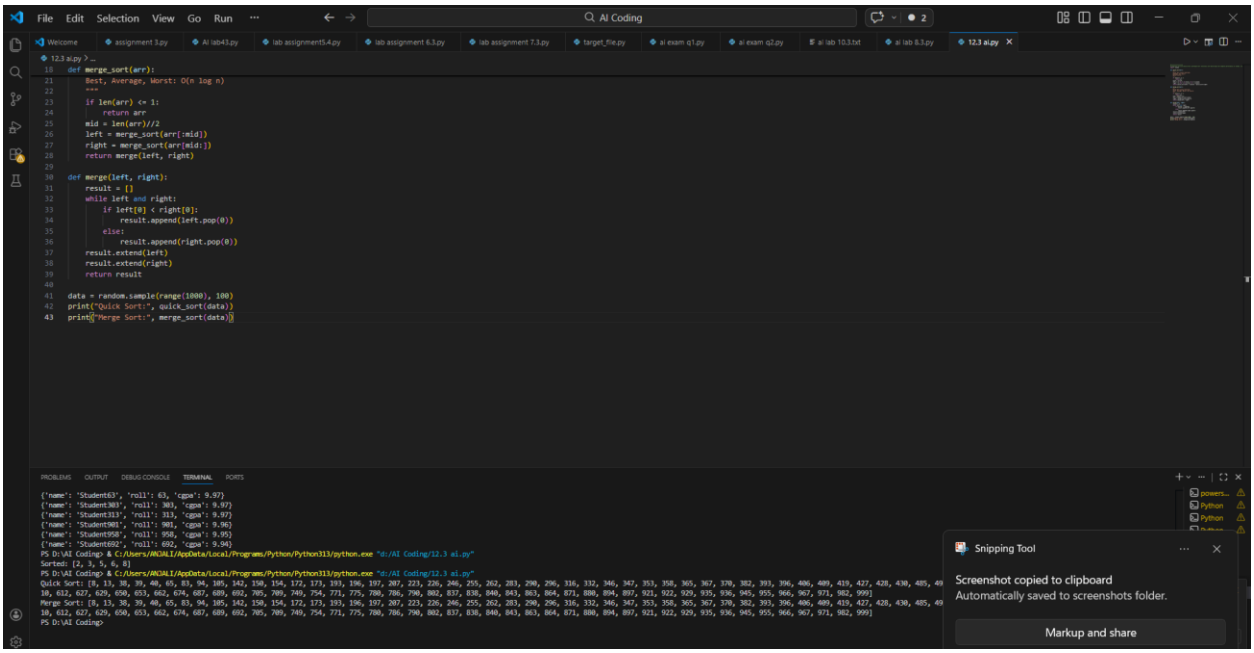
- **Task: Write a Python implementation of Bubble Sort**



**Task 3: Quick Sort and Merge Sort Comparison**

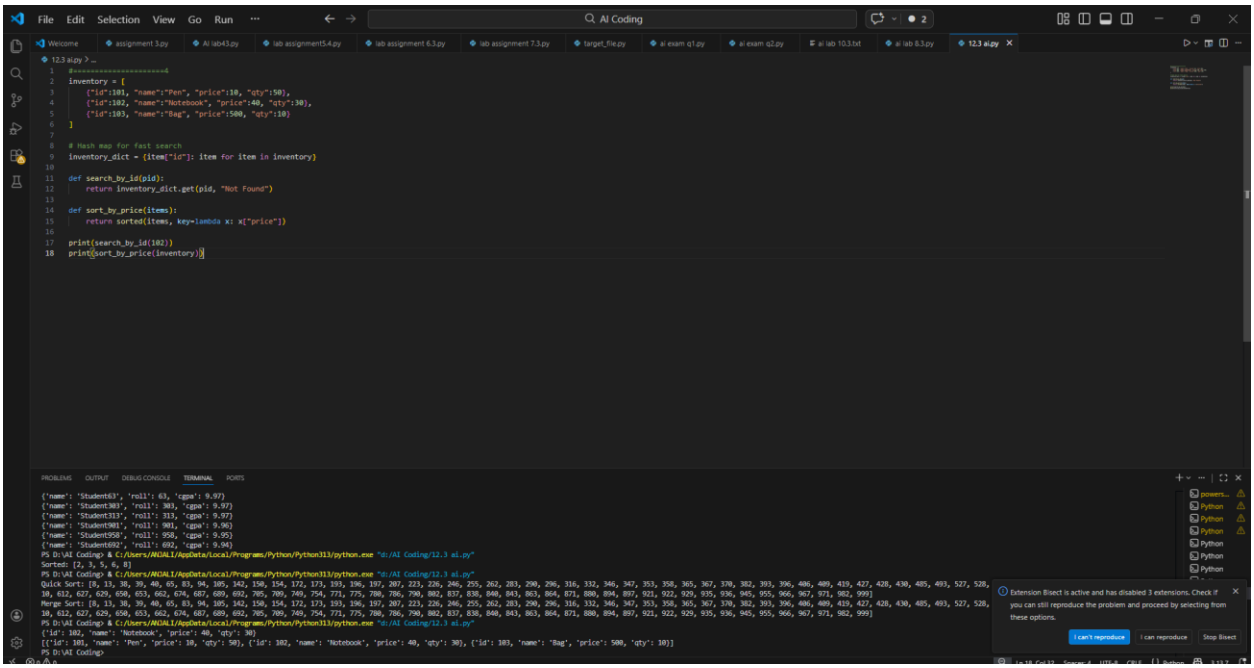
- **Task: Implement Quick Sort and Merge Sort using recursion**



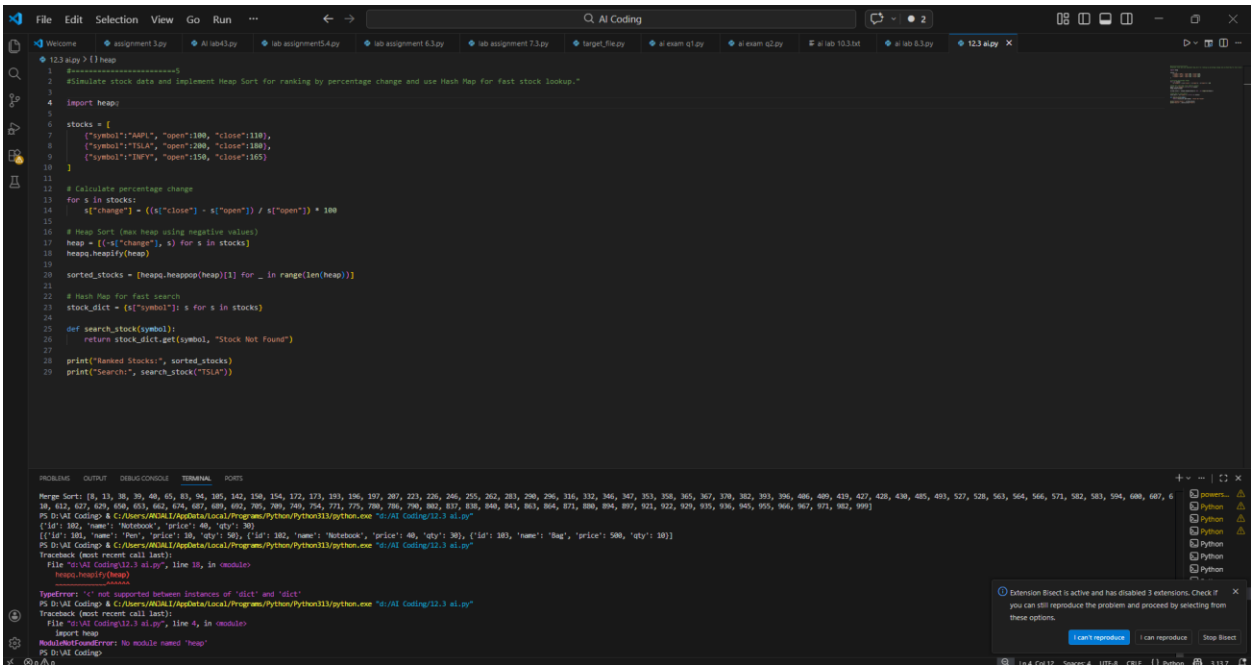



**Task 4 (Real-Time Application – Inventory Management System)**

**Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff**



**Task 5: Real-Time Stock Data Sorting & Searching**

**Scenario:**

**An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.**